MITTUNIVERSITETET

Laboration: Labb 3 SDN

Laborant: Hussein Alshaikhshaker
StudentID: hual2302

Kurs: DT222G

Datum: 2025-11-26

Installation Mininet & POX

Mininet

Installation av Mininet installerades lyckligt utan problem. Det installerades med kommandot angivet i laborationsinstruktionerna.

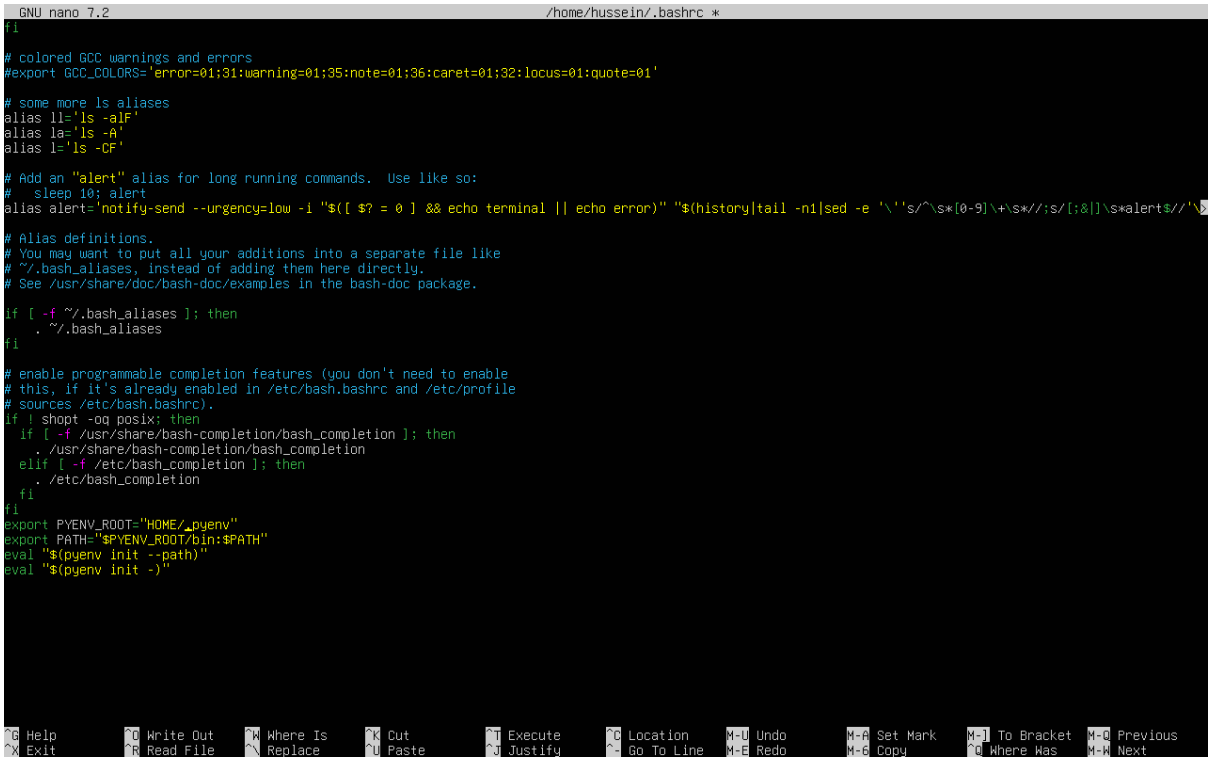
POX

POX installerades med kommandot [git clone], klonade repot som finns i ”https://github.com/noxrepo/pox”.

Problem

Jag följde instruktionerna under appendix A i labbinstruktionerna, det enda problemet jag mötte på var när jag skulle sätta upp python miljön, jag råkade skriva fel i PATH i konfigurationsfilen för bash (~/.bashrc).

För att lösa detta öppnade jag filen /bashrc med kommandot [nano] och lade till ett mellanslag som jag hade missat i PATH.



Figur 1: Detta är konfigurationsfilen för bash, efter att jag la till ett mellanslag mellan HOME/ och .pyenv

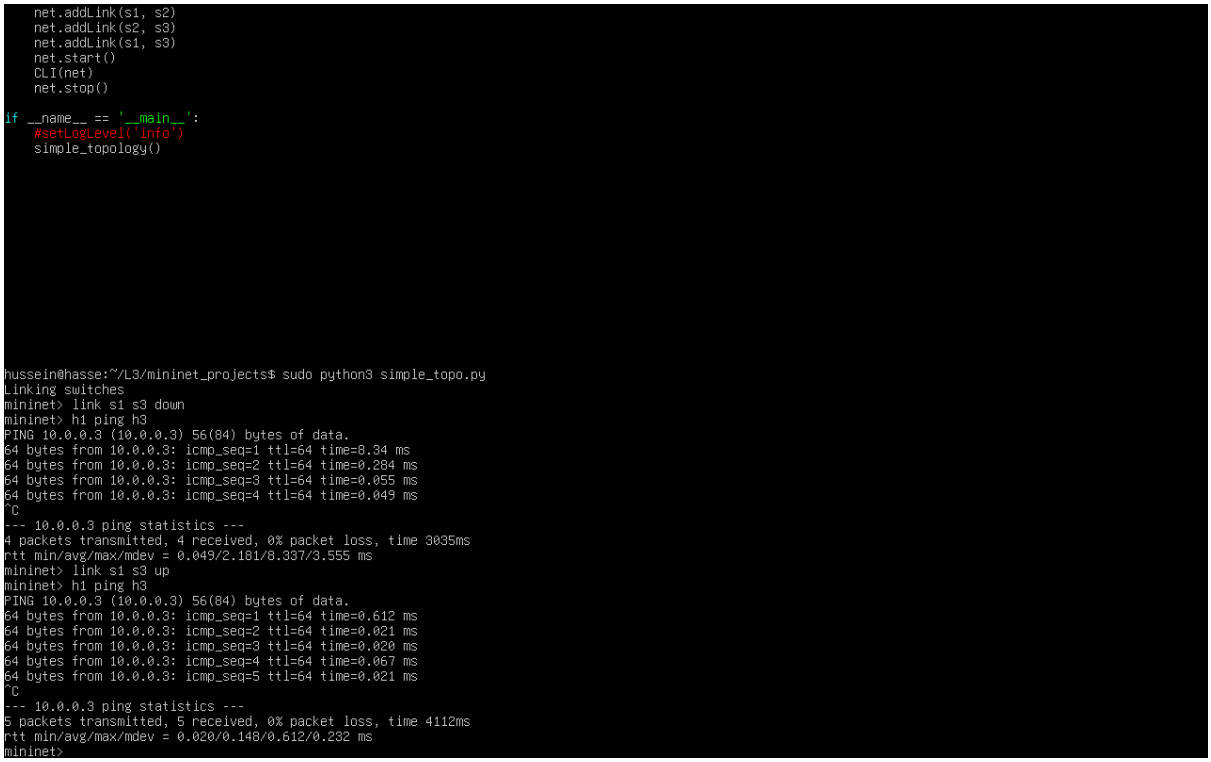
Efter detta steg kunde jag starta python-miljön med äldre version som var kompatibel med POX 0.7.0.



Figur 2: Bilden visar vilken python-tolk som används i POX mappen

Byggandet av ett mini-nätverk

Python koden finns i appendix A.

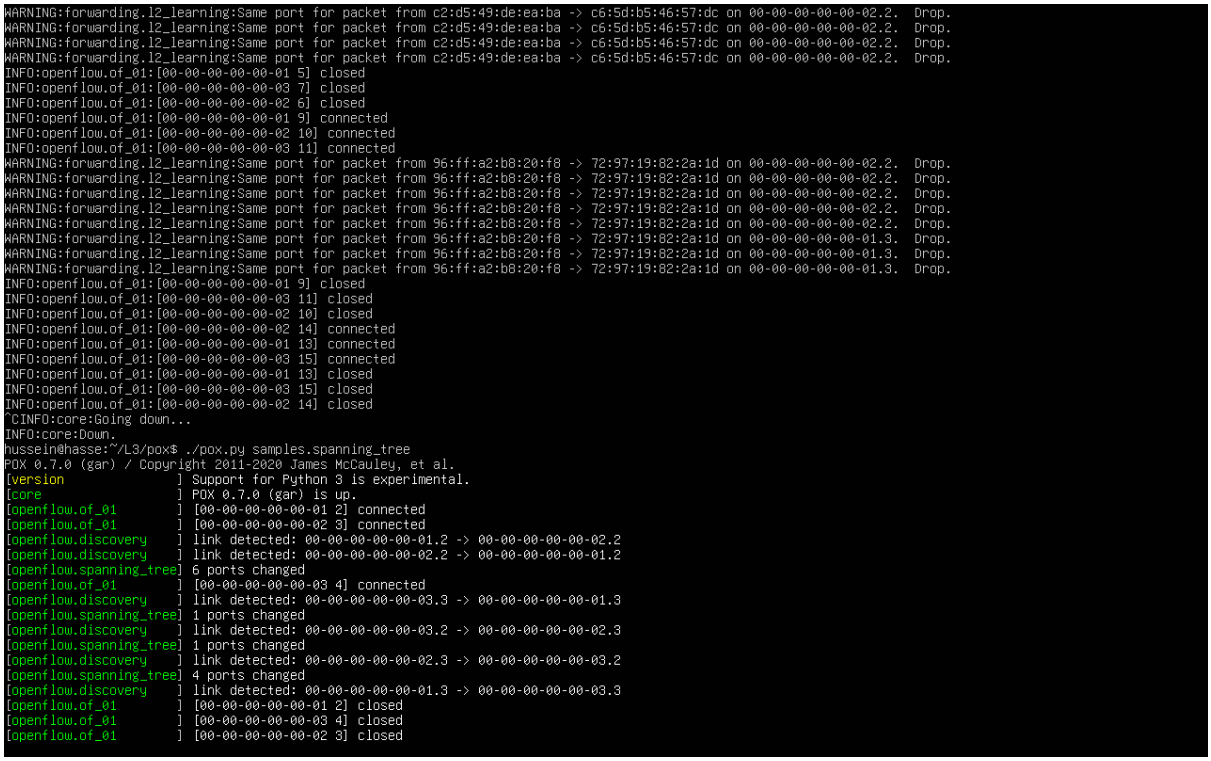


Figur 3: Denna skärmdump visar kommandot [ping]

I figur 3 ser man skillnaden mellan att ha en länk mellan switch 1 och switch 3. Skillnaden är RTT (Round Trip Time). Genom att köra kommandot [link s1 s3 up], får man mindre tid att skicka och ta emot paket mellan switch 1 och switch 3. Notera att när jag hade länken mellan s1 och s3 aktiverad, skickades SIGINT efter fem paket skickade. Medan i första anropet då länken var inaktiverad hann den skicka endast fyra paket innan den fick stoppsignalen. Därför visar RTT högre värde i andra anropet, annars hade totala tiden för inaktiverad länk visat högre värde än inaktiverad länk. Man kan annars se att per paket visar aktiverad länk mindre tid. Dock tyckte jag det var märkligt i fjärde paketet där inaktiverad länk visar 0.049ms och aktiverad länk visar 0.067ms, jag vet inte riktigt varför. Kan vara på grund av att jag sitter i virtuell miljö.

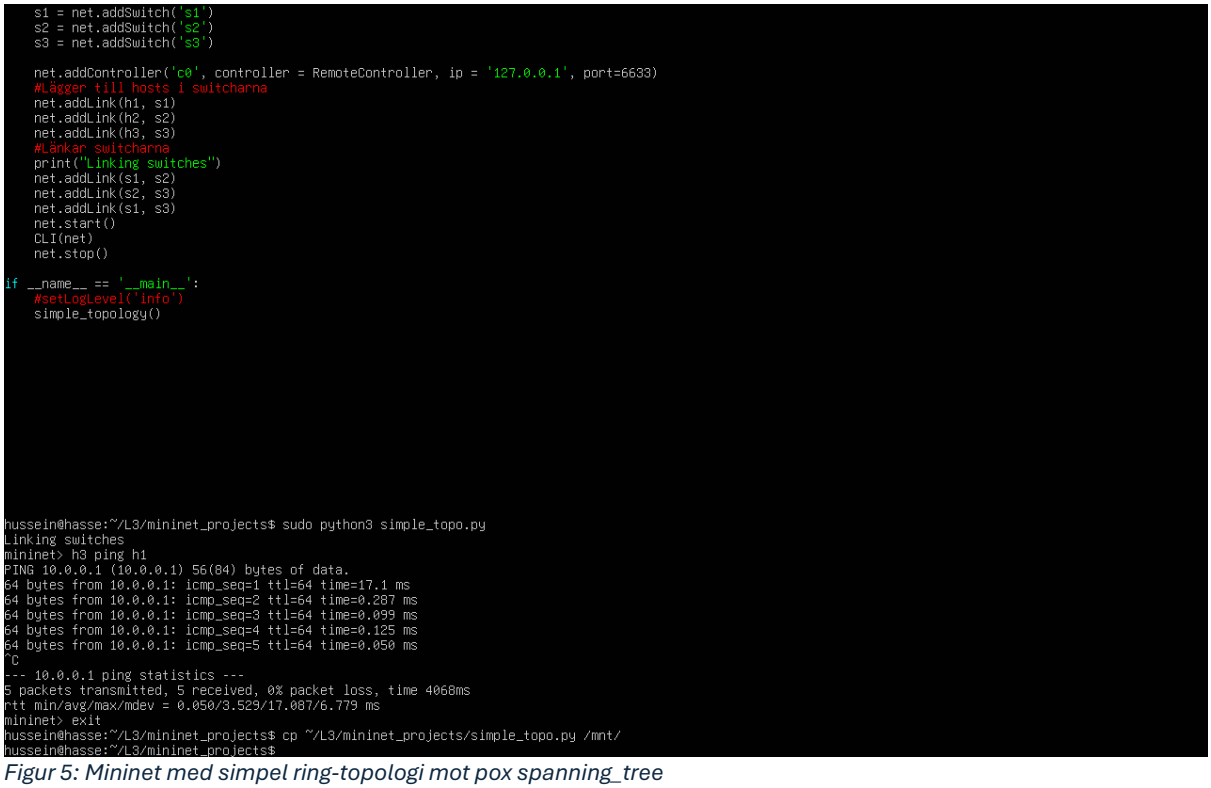
Hur som helst så sjunker RTT i och med att vi har en direkt sluten väg mellan s1 och s3, data skickas direkt från switch till switch, när länken är nere måste controllern välja en annan väg där den i detta fall går via s2 och där med tar det längre tid.

Remote SDN Kontroller



Figur 4: pox kontroller med spanning_tree komponenten

I figur 4 ser man POX kontroller är i gång med komponenten spanning_tree.



Figur 5: Mininet med simpel ring-topologi mot pox spanning_tree

I figur 5 ser man att jag har kopplat s1 och s3 [net.addLink(s1, s3)] och precis efter det körde jag i gång skriptet.

Min mininet-topologi, kopplad till POX med L2_learning, visade tydligt att nätverket inte fungerade korrekt när det fanns en cirkulär topologi. När jag lade till en extra länk mellan s1 och s3 uppstod en loop, vilket resulterade i 100% paketförlust eftersom lager-2-controller inte kan hantera loopar.

När samma topologi, i stället kopplades till POX med spanning_tree-komponenten kunde jag däremot pinga mellan hosts utan problem. Spanning_tree modulen byggde automatiskt ett träd av nätverket och blockerade portar som skulle skapa loopar.

Assignment

Byggandet av nätverket

Python skriptet för nätverket finns i appendix C.

Justerad POX komponent

Översikt

Python koden finns i appendix D.

Jag ändrade klassnamn på l2_learning till noLoopController och LearningSwitch till preventLoopLearningSwitch.

Min controller använder POX-modulen Discovery för att få information om vilka länkar som finns mellan switcharna. Varje gång en länk antingen upptäcks eller tas bort triggar POX ett LinkEvent. Jag har inte ändrat så mycket i den ursprungliga l2_learning, jag lade till endast switch-kopplingar i _handle_ConnectionUp för att lagra kopplingsobjekt vid anslutning mellan kontroller och switch och lade till funktionen _handle_LinkEvent som hanterar länk-event. Min funktion _handle_LinkEvent ansvarar för att uppdatera min interna topologi-mappning samt avgöra när porten mellan S3 och S3 ska stängas eller öppnas.

Min kod fungerade inte först, kunde inte pinga mellan hosts. Först hade jag d1, p1, d2, p2=link.dpid1, link.port1... Jag valde att använda inbyggda funktionen str_to_dpid för att konvertera DPID (00-00-00-00-00-03) till 3, detta säkerställer att jag jämför med heltal.

Mappning

Jag använder en datastruktur av typen:

switch_ports = {
S1 {2:2, 4:3}
S2 {1:2, ..}
...
}

Detta innebär att s1 går till s2 via port 2, och s1 går till s4 via port 3. S2 går till s1 via port 2...

Med hjälp av denna mappning kan min kontroller alltid veta vilka portar som används mellan switcharna, och därmed kan jag manipulera switchar och portar som jag vill.

Upptäckt av ny länk (event.added)

När en länk kommer upp gör min kontroller följande:

Loggar vilka switchar och portar involverade.
Den lagrar kopplingen åt båda hållen.
Checka om S3 är kopplad till både S2 och S4, i så fall;
Lagra vilken port det är som går från s3 till s2 i en variabel (s3_port_to_s2).
Hämta kopplingen mellan kontrollern och switchen.
Om det finns koppling;
Försök;
Skapa port-mod meddelande.
Sätter vilka portar meddelandet ska appliceras på.
Stäng porten/portarna.
Skicka meddelandet.
Logga vilken port/portar som stängdes ned.
Annars;
Kasta exception.

Länk läggs ned (event.removed)

När en länk läggs ned gör min kontroller följande:

Loggar vilka switchar och portar involverade.
Uppdatera min mappning.
Om länken som läggs ned är S3<->S4;
Försök;
Skapa port-mod meddelande.
Hämta porten mellan s3 och s2.
Rensa alla config-flaggor.
Sätt vilken flagga som ska påverkas.
Skicka meddelandet till switch.
Port s3->s2 sätts upp igen.
Logga info.
Annars;
Kasta exception.

Sedan försök flusha-flöden för den porten;
Skapa ett ofp_flow_mod meddelande med kommandot OFPFC_DELETE, ta bort matchande flöden.
Alla flöden som använder den porten tas bort.
Skicka flush-meddelande. (Flusha alla flöden för porten).

Logga att flödena är rensade.

Switch-meddelanden (OpenFlow)

I min labb kommunicerar kontrollern med switcharna via OpenFlow-meddelanden. Jag använder ofp_port_mod och ofp_flow_mod för att ändra portkonfigurationen och lägga till, ändra eller ta bort flöden i switchens flödes-tabell. Detta möjliggör för mig att förebygga loopar i min ring-topologi.

Exempel från min kod:
[msg = of.ofp_port_mod()]

Detta skapar ett nytt OpenFlow-portmeddelande som skickas till switchen.
Sedan väljer jag vilka portar på den switchen som ska påverkas, och konfigurerar switchen med [msg.config].
Jag satte [ms.config=of.OFPPC_PORT_DOWN] för att stänga porten. Sedan satte jag masken till down också.

När portar öppnas/stängs kan gamla flöden fortfarande finnas i switchen. Dessa måste tas bort för att undvika felaktig forwarding. För att komma runt detta problem kan man flusha (Rensa) gamla flöden med [of.OFPFC_DELETE], detta tar bort matchande flöden.

När man har anpassat meddelandet till det önskade ändamålet så skickar man meddelandet till switchen med [koppling.send(meddelande)], där ”koppling” är kopplingsobjektet mellan switch och kontroller man har hämtat (Jag skapar nästlad-dictionary i konstruktorn och hämtar det i _handle_ConnectionUP) och ”meddelande” är det meddelandet man anpassat.

Sammanfattning

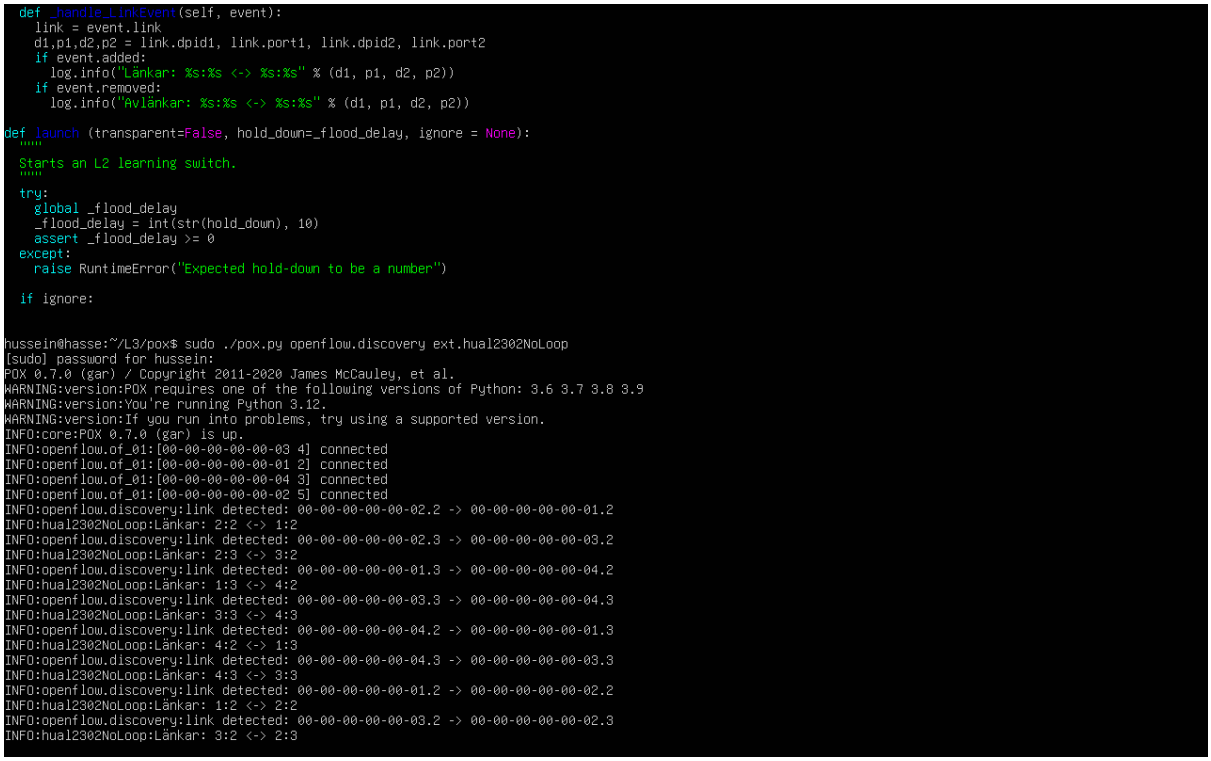
Min custom POX-kontroller noLoopController har modifierats med att lägga till funktionalitet för att undvika loopar i ring-topologi nätverk. Kontrollern använder POX-modulen openflow.discovery för att upptäcka nätverkstopologi och reagerar på förändringar i länkstatus.

Verifiering

I bilderna nedan verifieras komponenten. Min komponent resulterade i att inga loopar sker. Porten S3<->S2 stängdes automatiskt när S3 var kopplad till både S2 och S4, vilket förebygger loopar. När länken S3<->S4 tas bort öppnas porten S3<->S2 igen och gamla flöden rensas med ofp_flow_mod.

RTT mellan hosts minskar när direktlänkar används.

Flöden i switchar matchar alltid den aktuella topologin, vilket verifierar korrekt funktion av både port-modifiering och flow-flush.



```
  def _handle_LinkEvent(self, event):
    link = event.link
    d1,p1,d2,p2 = link.dpid1, link.port1, link.dpid2, link.port2
    if event.added:
      log.info("Länkar: %s:%s <-> %s:%s" % (d1, p1, d2, p2))
    if event.removed:
      log.info("Avlänkar: %s:%s <-> %s:%s" % (d1, p1, d2, p2))

def launch (transparent=False, hold_down=_flood_delay, ignore = None):
  """
  Starts an L2 learning switch.
  """
  try:
    global _flood_delay
    _flood_delay = int(str(hold_down), 10)
    assert _flood_delay >= 0
  except:
    raise RuntimeError("Expected hold-down to be a number")

  if ignore:


hussein@hasse:~/L3/pox$ sudo ./pox.py openflow.discovery ext.hual2302NoLoop
[sudo] password for hussein:
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-03 4] connected
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:openflow.of_01:[00-00-00-00-00-04 3] connected
INFO:openflow.of_01:[00-00-00-00-00-02 5] connected
INFO:openflow.discovery:link detected: 00-00-00-00-00-02.2 -> 00-00-00-00-00-01.2
INFO:hual2302NoLoop:Länkar: 2:2 <-> 1:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-02.3 -> 00-00-00-00-00-03.2
INFO:hual2302NoLoop:Länkar: 2:3 <-> 3:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-01.3 -> 00-00-00-00-00-04.2
INFO:hual2302NoLoop:Länkar: 1:3 <-> 4:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-03.3 -> 00-00-00-00-00-04.3
INFO:hual2302NoLoop:Länkar: 3:3 <-> 4:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-04.2 -> 00-00-00-00-00-01.3
INFO:hual2302NoLoop:Länkar: 4:2 <-> 1:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-04.3 -> 00-00-00-00-00-03.3
INFO:hual2302NoLoop:Länkar: 4:3 <-> 3:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-01.2 -> 00-00-00-00-00-02.2
INFO:hual2302NoLoop:Länkar: 1:2 <-> 2:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-03.2 -> 00-00-00-00-00-02.3
INFO:hual2302NoLoop:Länkar: 3:2 <-> 2:3
```

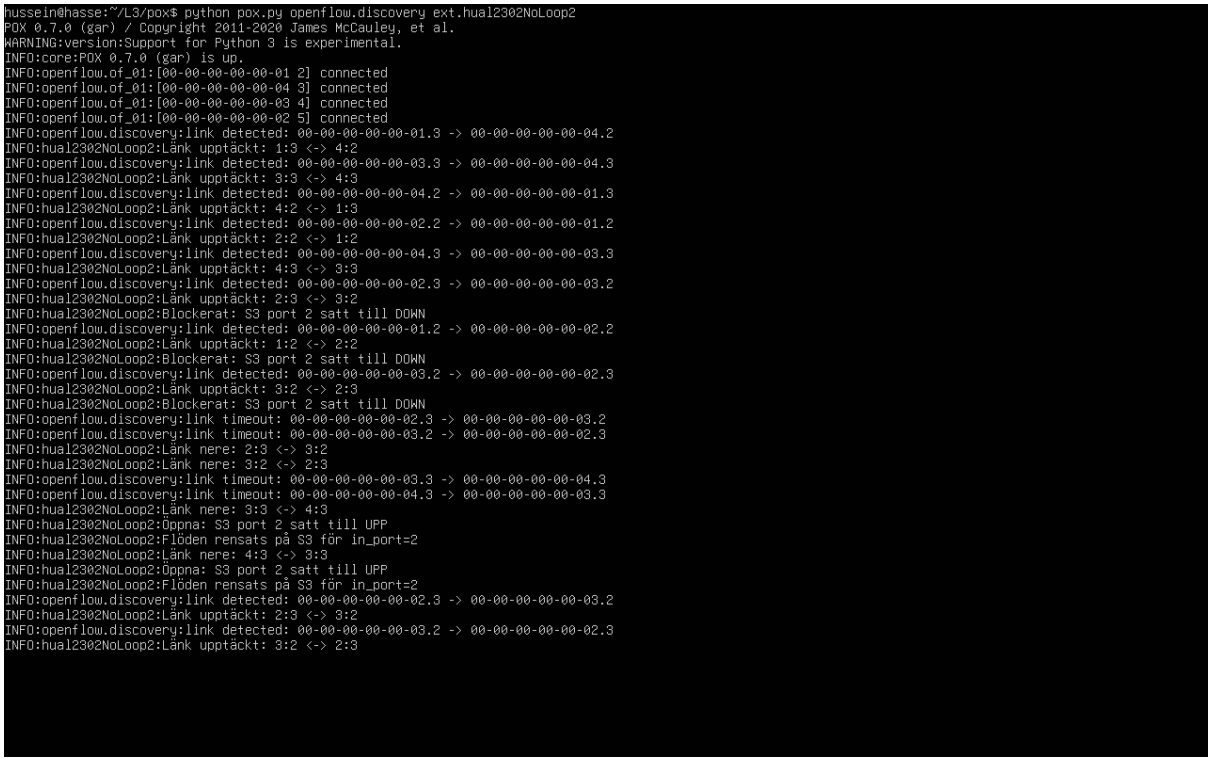
Figur 6: Denna bild visar hur min topologi är uppbyggd

```
hussein@hasse:~/L3/pox$ python pox.py openflow.discovery ext.hual2302NoLoop2
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:openflow.of_01:[00-00-00-00-00-04 3] connected
INFO:openflow.of_01:[00-00-00-00-00-03 4] connected
INFO:openflow.of_01:[00-00-00-00-00-02 5] connected
INFO:openflow.discovery:link detected: 00-00-00-00-00-01.3 -> 00-00-00-00-00-04.2
INFO:hual2302NoLoop2:Länk upptäckt: 1:3 <-> 4:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-03.3 -> 00-00-00-00-00-04.3
INFO:hual2302NoLoop2:Länk upptäckt: 3:3 <-> 4:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-04.2 -> 00-00-00-00-00-01.3
INFO:hual2302NoLoop2:Länk upptäckt: 4:2 <-> 1:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-02.2 -> 00-00-00-00-00-01.2
INFO:hual2302NoLoop2:Länk upptäckt: 2:2 <-> 1:2
INFO:openflow.discovery:link detected: 00-00-00-00-00-04.3 -> 00-00-00-00-00-03.3
INFO:hual2302NoLoop2:Länk upptäckt: 4:3 <-> 3:3
INFO:openflow.discovery:link detected: 00-00-00-00-00-02.3 -> 00-00-00-00-00-03.2
INFO:hual2302NoLoop2:Länk upptäckt: 2:3 <-> 3:2
INFO:hual2302NoLoop2:Blockerat: S3 port 2 satt till DOWN
INFO:openflow.discovery:link detected: 00-00-00-00-00-01.2 -> 00-00-00-00-00-02.2
INFO:hual2302NoLoop2:Länk upptäckt: 1:2 <-> 2:2
INFO:hual2302NoLoop2:Blockerat: S3 port 2 satt till DOWN
INFO:openflow.discovery:link detected: 00-00-00-00-00-03.2 -> 00-00-00-00-00-02.3
INFO:hual2302NoLoop2:Länk upptäckt: 3:2 <-> 2:3
INFO:hual2302NoLoop2:Blockerat: S3 port 2 satt till DOWN
INFO:openflow.discovery:link timeout: 00-00-00-00-00-02.3 -> 00-00-00-00-00-03.2
INFO:openflow.discovery:link timeout: 00-00-00-00-00-03.2 -> 00-00-00-00-00-02.3
INFO:hual2302NoLoop2:Länk nere: 2:3 <-> 3:2
INFO:hual2302NoLoop2:Länk nere: 3:2 <-> 2:3
```

Figur 7: Här ser man att min kontroller tar ned länken mellan s2 och s3 direkt

```
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> h4 ping h3
PING 10.0.9.3 (10.0.9.3) 56(84) bytes of data.
64 bytes from 10.0.9.3: icmp_seq=1 ttl=64 time=28.5 ms
64 bytes from 10.0.9.3: icmp_seq=2 ttl=64 time=0.250 ms
^C
--- 10.0.9.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.250/14.393/28.537/14.143 ms
mininet> link s3 s4 down
mininet> h4 ping h3
PING 10.0.9.3 (10.0.9.3) 56(84) bytes of data.
64 bytes from 10.0.9.3: icmp_seq=1 ttl=64 time=27.6 ms
64 bytes from 10.0.9.3: icmp_seq=2 ttl=64 time=0.180 ms
64 bytes from 10.0.9.3: icmp_seq=3 ttl=64 time=0.034 ms
^C
--- 10.0.9.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2033ms
rtt min/avg/max/mdev = 0.034/9.284/27.639/12.978 ms
mininet> h4 ping h1
PING 10.0.9.1 (10.0.9.1) 56(84) bytes of data.
64 bytes from 10.0.9.1: icmp_seq=1 ttl=64 time=25.5 ms
64 bytes from 10.0.9.1: icmp_seq=2 ttl=64 time=0.211 ms
64 bytes from 10.0.9.1: icmp_seq=3 ttl=64 time=0.052 ms
^C
--- 10.0.9.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 0.052/8.591/25.510/11.963 ms
mininet> h3 ping h2
PING 10.0.9.2 (10.0.9.2) 56(84) bytes of data.
64 bytes from 10.0.9.2: icmp_seq=1 ttl=64 time=17.7 ms
64 bytes from 10.0.9.2: icmp_seq=2 ttl=64 time=0.207 ms
64 bytes from 10.0.9.2: icmp_seq=3 ttl=64 time=0.082 ms
64 bytes from 10.0.9.2: icmp_seq=4 ttl=64 time=0.043 ms
^C
--- 10.0.9.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3031ms
rtt min/avg/max/mdev = 0.043/4.506/17.695/7.614 ms
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figur 8: Här ser man att jag sätter ned länken mellan s4 och s3, och kan fortsätta pinga mellan alla hosts



Figur 9: Länken mellan s3 och s2 tas upp igen

Appendix A

```
from mininet.net import Mininet
from mininet.node import OVSController
from mininet.cli import CLI
from mininet.log import setLogLevel

def simple_topology():
    net = Mininet(controller = OVSController)
    #Skapar hosts i nätet
    h1 = net.addHost('h1')
    h2 = net.addHost('h2')
    h3 = net.addHost('h3')

    #Lägger till switchar i nätet
    s1 = net.addSwitch('s1')
    s2 = net.addSwitch('s2')
    s3 = net.addSwitch('s3')

    c0 = net.addController('c0')
    #Lägger till hosts i switcharna
    net.addLink(h1, s1)
    net.addLink(h2, s2)
    net.addLink(h3, s3)
    #Länkar switcharna
    print("Linking switches")
    net.addLink(s1, s2)
    net.addLink(s2, s3)
    net.addLink(s1, s3)
    net.start()
    CLI(net)
    net.stop()

if __name__ == '__main__':
    #setLogLevel('info')
    simple_topology()
```

Appendix B

```
from mininet.net import Mininet
from mininet.node import OVSController
from mininet.cli import CLI
from mininet.log import setLogLevel
from mininet.node import RemoteController

def simple_topology():
    net = Mininet(controller = None)
    #Skapar hosts i nätet
    h1 = net.addHost('h1')
    h2 = net.addHost('h2')
    h3 = net.addHost('h3')

    #Lägger till switchar i nätet
    s1 = net.addSwitch('s1')
    s2 = net.addSwitch('s2')
    s3 = net.addSwitch('s3')

    net.addController('c0', controller = RemoteController, ip = '127.0.0.1', port=6633)
    #Lägger till hosts i switcharna
    net.addLink(h1, s1)
    net.addLink(h2, s2)
    net.addLink(h3, s3)
    #Länkar switcharna
    print("Linking switches")
    net.addLink(s1, s2)
    net.addLink(s2, s3)
    net.addLink(s1, s3)
    net.start()
    CLI(net)
    net.stop()

if __name__ == '__main__':
    #setLogLevel('info')
    simple_topology()
```

Appendix C

```
from mininet.net import Mininet
from mininet.node import OVSController
from mininet.cli import CLI
from mininet.log import setLogLevel
from mininet.topo import Topo
from mininet.node import RemoteController
#Klass för labbens topologi
class fourSwitchTopo(Topo):
  #Funktion som bygger topologin
  def build(self):
    #Skapar lägger till switchar i topologin
    print("Skapar switches")
    switches=["s1", "s2", "s3", "s4"]
    for sw in switches:
      self.addSwitch(sw)
    #Skapar hosts och lägger till dem i topon
    print("Skapar hosts")
    hosts = ["h1", "h2", "h3", "h4"]
    for h in hosts:
      self.addHost(h)
    #Länkar switcharna med hosts
    print("Länkar hosts och switches")
    self.addLink(hosts[0], switches[0])
    self.addLink(hosts[1], switches[3])
    self.addLink(hosts[2], switches[1])
    self.addLink(hosts[3], switches[2])

    #Länkar switcharna
    print("Länkar switcharna med varandra")
    self.addLink(switches[0], switches[1]) #s1 <-> s2
    self.addLink(switches[0], switches[3]) #s1 <-> s4
    self.addLink(switches[2], switches[1]) #s3 <-> s2
    self.addLink(switches[2], switches[3]) #s3 <-> s4

def main():
  setLogLevel('info')
  net = Mininet(topo=fourSwitchTopo(), controller=lambda name:
RemoteController(name, ip='127.0.0.1', port=6633), ipBase='10.0.9.0/24')
  net.start()
  #sätter Ips på hosts
  net.get('h1').setIP('10.0.9.1/24')
  net.get('h2').setIP('10.0.9.2/24')
  net.get('h3').setIP('10.0.9.3/24')
  net.get('h4').setIP('10.0.9.4/24')
```

```
    CLI(net)
    net.stop()

if __name__ == '__main__':
    main()
```

Appendix D

```
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.util import dpid_to_str, str_to_dpid
from pox.lib.util import str_to_bool
import pox.openflow.discovery
from pox.openflow.discovery import Discovery
import time

log = core.getLogger()

# We don't want to flood immediately when a switch connects.
# Can be overriden on commandline.
_flood_delay = 0

class preventLoopLearningSwitch (object):
  def __init__ (self, connection, transparent):
    # Switch we'll be adding L2 learning switch capabilities to
    self.connection = connection
    self.transparent = transparent

    # Our table
    self.macToPort = {}

    # We want to hear PacketIn messages, so we listen
    # to the connection
    connection.addListeners(self)

    # We just use this to know when to log a helpful message
    self.hold_down_expired = _flood_delay == 0

    #log.debug("Initializing LearningSwitch, transparent=%s",
    #          str(self.transparent))

  def _handle_PacketIn (self, event):
    """
    Handle packet in messages from the switch to implement above algorithm.
    """
```

```
packet = event.parsed

def flood (message = None):
 """ Floods the packet """
 msg = of.ofp_packet_out()
 if time.time() - self.connection.connect_time >= _flood_delay:
  # Only flood if we've been connected for a little while...

  if self.hold_down_expired is False:
   # Oh yes it is!
   self.hold_down_expired = True
   log.info("%s: Flood hold-down expired -- flooding",
      dpid_to_str(event.dpid))

  if message is not None: log.debug(message)
  #log.debug("%i: flood %s -> %s", event.dpid,packet.src,packet.dst)
  # OFPP_FLOOD is optional; on some switches you may need to change
  # this to OFPP_ALL.
  msg.actions.append(of.ofp_action_output(port = of.OFPP_FLOOD))
 else:
  pass
  #log.info("Holding down flood for %s", dpid_to_str(event.dpid))
 msg.data = event.ofp
 msg.in_port = event.port
 self.connection.send(msg)

def drop (duration = None):
 """
 Drops this packet and optionally installs a flow to continue
 dropping similar ones for a while
 """
 if duration is not None:
  if not isinstance(duration, tuple):
   duration = (duration,duration)
  msg = of.ofp_flow_mod()
  msg.match = of.ofp_match.from_packet(packet)
  msg.idle_timeout = duration[0]
  msg.hard_timeout = duration[1]
  msg.buffer_id = event.ofp.buffer_id
  self.connection.send(msg)
```

```
    elif event.ofp.buffer_id is not None:
      msg = of.ofp_packet_out()
      msg.buffer_id = event.ofp.buffer_id
      msg.in_port = event.port
      self.connection.send(msg)

    self.macToPort[packet.src] = event.port # 1

    if not self.transparent: # 2
      if packet.type == packet.LLDP_TYPE or packet.dst.isBridgeFiltered():
        drop() # 2a
        return

    if packet.dst.is_multicast:
      flood() # 3a
    else:
      if packet.dst not in self.macToPort: # 4
        flood("Port for %s unknown -- flooding" % (packet.dst,)) # 4a
      else:
        port = self.macToPort[packet.dst]
        if port == event.port: # 5
          # 5a
          log.warning("Same port for packet from %s -> %s on %s.%s.  Drop."
              % (packet.src, packet.dst, dpid_to_str(event.dpid), port))
          drop(10)
          return
        # 6
        log.debug("installing flow for %s.%i -> %s.%i" %
                  (packet.src, event.port, packet.dst, port))
        msg = of.ofp_flow_mod()
        msg.match = of.ofp_match.from_packet(packet, event.port)
        msg.idle_timeout = 10
        msg.hard_timeout = 30
        msg.actions.append(of.ofp_action_output(port = port))
        msg.data = event.ofp # 6a
        self.connection.send(msg)


class noLoopController (object):
  """
  Waits for OpenFlow switches to connect and makes them learning switches.
```

```
  """
  def __init__ (self, transparent, ignore = None):
    core.openflow.addListeners(self)
    core.openflow_discovery.addListeners(self)
    self.transparent = transparent
    self.ignore = set(ignore) if ignore else ()
    self.switch_ports={} #Nästlad-dictionary, hjälper till att hålla koll hur kopplingen sker mellan switchar och portar.
    self.switch_connections={} #Denna hjälper min kontroller att prata med specifika switches med DPID.
    self.s3_port_to_s2=None

  def _handle_ConnectionUp (self, event):
    self.switch_connections[event.dpid] = event.connection #spara connection objekt när en switch ansluter sig.
    if event.dpid in self.ignore:
      log.debug("Ignoring connection %s" % (event.connection,))
      return
    log.debug("DEBUG ConnectionUp: dpid=%s (%s)", event.dpid, dpid_to_str(event.dpid))
    preventLoopLearningSwitch(event.connection, self.transparent)

  #Hantera länk evet från discovery
  def _handle_LinkEvent(self, event):
    link=event.link
    #Säkerställ att DPID behandlas som heltal
    try:
      d1 = str_to_dpid(link.dpid1) if isinstance(link.dpid1, str) else link.dpid1
      d2 = str_to_dpid(link.dpid2) if isinstance(link.dpid2, str) else link.dpid2
    except:
      d1, d2 = link.dpid1, link.dpid2
    p1 = link.port1
    p2 = link.port2
    #Skapa entries för swithcarna om de inte finns
    self.switch_ports.setdefault(d1, {})
    self.switch_ports.setdefault(d2, {})
    #Om länk upptäcks
    if event.added:
      log.info("Länk upptäckt: %s:%s <-> %s:%s", d1,p1,d2,p2)
      #Uppdatera topo-mappningen åt båda hållen
      self.switch_ports[d1][d2] = p1
```

```
    self.switch_ports[d2][d1] = p2
    #Om s3 är kopplad mot s4 OCH s2, stäng porten mot s2.
    if(3 in self.switch_ports and 2 in self.switch_ports[3] and 4 in self.switch_ports[3]):
     self.s3_port_to_s2=self.switch_ports[3][2] #Detta är den exakta porten som går från
s3 till s2
     conn=self.switch_connections.get(3) #Hämta kopplingen POX <-> s3
     if conn:
      try:
       msg=of.ofp_port_mod() #Skapa port-mod meddelande.
       msg.port_no=self.s3_port_to_s2 #Anpassa på portar angivna här
       msg.hw_addr=conn.ports[self.s3_port_to_s2].hw_addr #Hämta portens MAC-
adress från kopplings-objektet
       msg.config=of.OFPPC_PORT_DOWN #Stäng port/portar
       msg.mask=of.OFPPC_PORT_DOWN
       conn.send(msg) #Skicka meddelandet till switchen
       log.info("Blockerat: S3 port %s satt till DOWN", self.s3_port_to_s2)
      except Exception as e:
       log.exception("Kan ej blockera port s3<->s2: %s", e)
   #Om länk tas bort
   elif event.removed:
    log.info("Länk nere: %s:%s <-> %s:%s", d1,p1,d2,p2)
    #Uppdatera min mappning
    self.switch_ports[d1].pop(d2, None)
    self.switch_ports[d2].pop(d1, None)
    #Om s3<->s4 down, ta upp länken s3<->s2
    if{d1, d2} == {3, 4} and self.s3_port_to_s2:
     conn = self.switch_connections.get(3)
     if conn:
      try:
       #öppna porten s3 till s2
       pm=of.ofp_port_mod()
       pm.port_no=self.s3_port_to_s2
       pm.hw_addr=conn.ports[self.s3_port_to_s2].hw_addr #MAC-adressen måste
med annars ignorerar switchen meddelandet
       pm.config=0 #Rensa alla config-flaggor
       pm.mask=of.OFPPC_PORT_DOWN #Just denna flagga ska påverkas
       conn.send(pm) #Skicka order till switch, effekt av detta är att port sätts upp igen
       log.info("Öppna: S3 port %s satt till UPP", self.s3_port_to_s2)
      except Exception as e:
       log.exception("Kan ej öppna port s3<->s2: %s", e)
      try: #Flusha flöden för den porten
```

```
        flush=of.ofp_flow_mod()
        flush.command=of.OFPFC_DELETE
        flush.match.in_port=of.ofp_match()
        flush.match.in_port=self.s3_port_to_s2
        conn.send(flush)
        log.info("Flöden rensats på S3 för in_port=%s", self.s3_port_to_s2)
       except Exception as e:
        log.exception("Kunde ej flusha flöden på S3: %s", e)

def launch (transparent=False, hold_down=_flood_delay, ignore = None):
 try:
  global _flood_delay
  _flood_delay = int(str(hold_down), 10)
  assert _flood_delay >= 0
 except:
  raise RuntimeError("Expected hold-down to be a number")

 if ignore:
  ignore = ignore.replace(',', ' ').split()
  ignore = set(str_to_dpid(dpid) for dpid in ignore)

 core.registerNew(noLoopController, str_to_bool(transparent), ignore)
```

Referenser

(u.d.). Hämtat från https://noxrepo.github.io/pox-doc/html/#openflow-spanning-tree

(u.d.). Hämtat från https://www.jha-tech.com/sv/news/what-is-the-difference-between-layer-2-and-layer-3-switches/

(u.d.). Hämtat från https://github.com/noxrepo/pox/blob/gar-experimental/pox/openflow/discovery.py

(u.d.). Hämtat från https://github.com/mininet/mininet/wiki/Introduction-to-Mininet

(u.d.). Hämtat från https://github.com/noxrepo/pox

(u.d.). Hämtat från https://noxrepo.github.io/pox-doc/html/